

Project Final Report

NeuroMove: An EMG-Based Smart Mobility Control System

Course: Capstone Design

Professor: Woongsup Kim

Team Name: 3ParkLee1

Team Members: Yeonwoo Lee / Sooyeon Park / Subeen Park / Won Park

Frontend	https://github.com/Capstone-EMG-3P1L/NeuroMove_frontend
Backend	https://github.com/Capstone-EMG-3P1L/NeuroMove_backend
AI	https://github.com/Capstone-EMG-3P1L/NeuroMove_AI
Arduino	https://github.com/Capstone-EMG-3P1L/NeuroMove_Arduino
Demonstration Video	https://github.com/Capstone-EMG-3P1L/NeuroMove_Materials/releases/tag/v1.0.0

Submission Date: June 18, 2026

1. General Description

The objective of this project is to design and implement a smart mobility control system that analyzes a user's movement intentions in real time using electromyography (EMG) signals and controls a mobility assistance device based on the inferred intentions.

The system collects muscle activity signals through EMG sensors and converts them into digital data using an ESP32 microcontroller before transmitting them to an AI server. The AI analysis module preprocesses the signals to remove noise and extract meaningful features, then employs a machine learning–based classification model to identify user intentions such as LEFT, RIGHT, STOP, and REST. The backend server manages user information, calibration results, device status, and operation logs while generating mobility control commands based on the AI inference results. In addition, when no response is detected for a certain period or abnormal patterns are identified, STOP or EMERGENCY_STOP commands are prioritized to ensure safe and reliable integration with the ESP32.

The significance of this system lies in the fact that it is not merely a signal measurement platform but a real-time intention inference system. By analyzing muscle activity to determine movement direction and prioritizing stop commands in hazardous situations, the system aims to provide a practical solution applicable to real-world mobility assistance systems.

Furthermore, considering that muscle characteristics vary among individuals, the system provides a calibration process and supports personalized control policies. Through these features, the project aims to establish a smart mobility control platform capable of operating reliably across diverse user environments.

2. Problem Statement

<Performance (Throughput and Response Time)>

An EMG-based control system must recognize user intentions in real time and therefore requires high responsiveness. Excessive delays between muscle contraction and control command generation can reduce usability and potentially compromise user safety.

Because the system involves multiple sequential stages—including signal acquisition, preprocessing, feature extraction, machine learning inference, and control command generation—processing latency at each stage must be minimized. To address this challenge, the proposed system adopts lightweight feature extraction techniques and an efficient inference architecture to support real-time operation.

<Data Accuracy and Reliability>

EMG signals are highly sensitive biological signals affected by sensor placement, skin condition, individual muscle characteristics, and muscle fatigue. In addition, electrical interference and environmental noise may degrade signal quality.

Such issues can reduce intention classification accuracy. Therefore, the system performs signal preprocessing, including DC offset removal and rectification, and extracts features such as Mean Absolute Value (MAV) and Root Mean Square (RMS) to generate stable input data. User-specific calibration is also incorporated to reflect individual physiological characteristics, thereby improving both data reliability and classification accuracy.

<Economic Feasibility (Cost)>

Many commercial biosignal-based control systems rely on expensive medical-grade equipment, limiting accessibility for general users. Furthermore, complex hardware configurations often increase maintenance costs.

To reduce system costs, this project utilizes relatively inexpensive MyoWare EMG sensors and ESP32 microcontrollers. The goal is to develop a cost-effective system that is suitable not only for research purposes but also for future commercialization.

<Control and Security>

Since mobility control systems directly affect user safety, a high level of reliability is essential. Incorrect intention recognition may result in unintended movement or collisions.

To prevent such situations, the proposed system maintains the REST state when signal strength remains below predefined thresholds and prioritizes STOP commands over all other movement commands. In addition, a reliable communication architecture is employed to prevent data loss or tampering during transmission. Device status and ESP32 connectivity are continuously monitored to ensure that control commands are processed safely under abnormal operating conditions.

<Efficiency>

Because EMG signals are continuously collected, large volumes of data are generated. Processing all data using computationally intensive algorithms could increase resource consumption and reduce system responsiveness.

To address this issue, the system employs computationally efficient features such as MAV and RMS to represent signals effectively. In addition, a lightweight machine learning model is adopted to ensure smooth operation in embedded environments.

<Service Usability and Scalability>

Providing an intuitive user experience is an important design consideration. The system should be easy to use with only a simple calibration procedure and should operate reliably across diverse user environments.

The platform is also designed with scalability in mind, allowing future applications beyond smart wheelchairs, including rehabilitation robots, prosthetic control systems, and smart healthcare devices. Planned service extensions include:

- Personalized control functions based on individual muscle characteristics
- User condition monitoring through muscle fatigue analysis
- Integration with smart wheelchairs and mobility-assistance robots
- Real-time healthcare services based on biosignal analysis
- Personalized rehabilitation support using accumulated user data

3. Preliminary Study & Need for the Project

3.1. Background

As societies continue to age and increasing attention is given to ensuring mobility rights for individuals with disabilities, the importance of mobility assistance technologies has grown significantly. Mobility devices such as powered wheelchairs play a crucial role in supporting independent living; however, most existing systems rely on joystick- or button-based interfaces, making them difficult to use for individuals with limited upper-limb strength or restricted hand movement.

Users affected by muscle atrophy, spinal cord injuries, or neurological disorders often experience difficulty performing the precise finger and wrist movements required to operate conventional mobility devices. Consequently, their freedom of movement is restricted, and they frequently become dependent on caregivers, reducing both autonomy and quality of life.

Human–Machine Interfaces (HMIs) based on biosignals have emerged as a promising solution to these challenges. In particular, Electromyography (EMG) measures electrical activity generated during muscle contraction and offers the advantage of rapidly and accurately identifying user movement intentions.

Recent advances in sensor technology and artificial intelligence have expanded the application of EMG-based systems across medical rehabilitation equipment, prosthetic control systems, and smart healthcare technologies. However, many existing studies rely on expensive equipment or operate only in controlled laboratory settings, limiting their practical deployment in everyday environments.

3.2. Current Scenario

Most mobility assistance devices, including powered wheelchairs, are currently controlled using physical interfaces such as joysticks and buttons. These methods assume precise hand and arm movements, making them less accessible for users with limited upper-limb functionality or neurological impairments. Although substantial research has been conducted on EMG-based control systems, many existing solutions depend on costly measurement equipment and closed research environments, limiting their applicability in daily life.

To address these limitations, NeuroMove was developed using low-cost MyoWare 2.0 EMG sensors and an ESP32 microcontroller, implementing the entire pipeline from signal acquisition and intention inference to mobility device control. The system consists of three independent repositories responsible for the AI server (Python/FastAPI), backend server (Java/Spring Boot), and frontend application (React/TypeScript).

In the current implementation, the ESP32 streams three-channel EMG data to the AI server through WebSocket communication. The AI server preprocesses the signals, extracts MAV and RMS features, and classifies four user intentions—LEFT, RIGHT, STOP, and REST—using a Random Forest model. The classification results are processed through REST-state handling and safety-prioritized STOP policies before being transmitted to the backend server. The backend then applies a Risk Score mechanism and fail-safe policies to determine final control commands, which are subsequently reflected in both the motor device and the web-based dashboard in real time.

As a result, the project has progressed beyond simple signal monitoring and successfully implemented an end-to-end system capable of performing actual mobility control.

3.3. Challenges

Several key technical challenges were encountered during the design and implementation of the proposed system.

- **Signal Quality and Individual Variability:** EMG signals are highly sensitive to sensor placement, skin condition, muscle fatigue, and electrical noise. Signal magnitude and distribution also vary considerably among users, making it difficult to apply uniform thresholds.
- **Real-Time Processing:** The latency introduced by signal acquisition, preprocessing, feature extraction, inference, and control command generation must be minimized to ensure prompt system response.
- **Safety:** Misclassification of user intentions can result in unintended movement or collisions. Therefore, multiple safety mechanisms are required to suppress movement when signals are weak or abnormal and to prioritize STOP commands.
- **Multi-Server Integration:** Because the AI server, backend server, frontend application, and embedded devices communicate through WebSocket and REST APIs, maintaining data consistency, communication reliability, and authentication is critical.

- **Device State Management:** A single ESP32 data stream must be dynamically routed between calibration and driving sessions, while abnormal disconnections must be detected and recovered automatically to prevent resource lock leakage.

3.4. Need for the project

Conventional mobility assistance devices primarily depend on joysticks, buttons, and manual control mechanisms, making intuitive operation difficult for users with limited upper-limb functionality. Furthermore, because physical capabilities differ among individuals, a single control method may not be equally suitable for all users. Therefore, there is a growing need for personalized mobility control technologies that reflect individual biosignals.

To address these limitations, this project proposes NeuroMove, an EMG-based smart mobility control system that utilizes low-cost EMG sensors and an ESP32-based embedded platform to collect muscle signals in real time and infer user movement intentions. When a user contracts specific muscles, the system analyzes the resulting EMG signals and converts them into mobility commands such as LEFT, RIGHT, STOP, and REST, which can then be used to control mobility assistance devices.

Beyond simple signal measurement, the system integrates AI-based intention classification with real-time control mechanisms to represent user intentions more accurately and reliably. To achieve this objective, a complete control pipeline incorporating EMG preprocessing, feature extraction, and machine learning–based intention inference has been developed.

Ultimately, the project aims to enable users with limited upper-limb functionality to control mobility devices more easily and intuitively while providing a foundational technology applicable to future smart wheelchairs, rehabilitation robots, and wearable healthcare devices. In addition, the project seeks to improve user autonomy and mobility convenience while demonstrating the practical feasibility of biosignal-based human–machine interface technologies.

4. Constraints

4.1. Technical Development Constraints

4.1.1 Hardware Operating Environment

The proposed system utilizes MyoWare 2.0 EMG sensors and an ESP32 microcontroller to acquire users' electromyography (EMG) signals. The EMG sensing module consists of three channels (ch1, ch2, and ch3) that measure electrical signals generated by muscle activity.

The ESP32 converts the acquired analog signals into digital data and transmits them to the AI server for further processing.

In addition, a motor control unit is employed to convert the final control commands generated by the backend server into actual motor operations. The motor controller receives commands such as FORWARD, LEFT, RIGHT, and STOP from the backend server and controls the rotation direction and operational state of DC motors through a motor driver board.

Accurate EMG signal acquisition requires consistent sensor placement and stable muscle conditions. Signal quality may be affected by sensor contact conditions and external noise sources. Furthermore, the stability of motor control depends on factors such as power supply conditions, wiring reliability, and motor driver performance.

4.1.2 AI Server Operating Environment

The AI server is developed using Python and the FastAPI framework. It processes EMG data received from the ESP32 in real time and performs signal preprocessing, feature extraction, and intention inference using a Random Forest classification model.

Because real-time processing is a critical requirement, server response time directly affects overall system performance. In addition, server resource consumption may increase when multiple users access the system simultaneously.

4.1.3 Backend Server Operating Environment

The backend server is implemented using Spring Boot and serves as the central management component of the system. It is responsible for handling user information, device information, session data, calibration results, AI inference outputs, and command logs. MySQL is used for persistent data storage and retrieval, while Redis is utilized for data requiring fast access, such as session management, token management, and device connection status monitoring.

The backend server processes inference results received from the AI server and communicates with both the motor control unit and the React-based dashboard. To ensure safe operation, it incorporates safety control mechanisms such as Risk Score evaluation, Finite State Machine (FSM) state management, and fail-safe policies that address delays, duplicated commands, and connection interruptions.

4.1.4 System Communication Environment

The system employs both WebSocket communication and REST APIs to support real-time EMG data transmission and mobility device control.

The ESP32 transmits EMG data collected from the MyoWare 2.0 sensors to the AI server through WebSocket communication. The AI server analyzes the received data and generates inference results, including movement intention, confidence score, signal quality, and fatigue score. These results are then forwarded to the backend server via REST APIs.

Based on the received inference results, the backend server determines the final control command and transmits it to the motor control unit through WebSocket communication. The React-based dashboard also communicates with the backend server using WebSocket, enabling real-time monitoring of AI inference results, motor status, risk levels, and FSM states.

Network delays or connection interruptions may affect the reliability of real-time intention recognition and motor control. Therefore, the communication architecture incorporates timestamps and WebSocket connection monitoring to maintain data ordering and verify connection status.

4.2. Economic Constraints

<Hardware Acquisition Costs>

The system requires MyoWare EMG sensors and ESP32 development boards for EMG signal acquisition. Additional costs arise from sensor electrodes, cables, and related components. The overall cost may increase if multiple sensors are required depending on the project scale.

<Server Operating Costs>

Operation of the AI server and backend server requires either cloud-based infrastructure or local testing servers. During the development phase, local environments are utilized to minimize expenses. However, cloud infrastructure costs may be incurred as the system scales and expands.

<Expansion Costs>

Future integration with actual smart wheelchairs or mobility assistance devices may require additional hardware components, including motor controllers, battery systems, and supplementary sensors. Furthermore, server expansion costs must be considered as the number of users increases.

4.3. Operational Constraints

<Development and Maintenance Costs>

Most software components used in this project are based on open-source technologies and therefore do not require additional licensing fees. However, in a future commercialization stage, additional expenses may arise from server operation, equipment maintenance, and user support services.

5. System Requirements

5.1. Functional Requirements

Function	Description
EMG Signal Acquisition	Collects users' EMG signals in real time using MyoWare sensors.
User Calibration	Measures individual muscle characteristics and generates baseline values and thresholds.
Signal Preprocessing	Performs DC offset removal and rectification to reduce signal noise.
Feature Extraction	Extracts features such as MAV and RMS for AI model input.
Intention Classification	Classifies user intentions into LEFT, RIGHT, STOP, and REST using the AI model.
Real-Time Data Transmission	Transfers data between the ESP32 and AI server through WebSocket communication.
Session Management	Provides functionality for creating and terminating user sessions.
Control Command Generation	Converts classified intentions into mobility control commands.
User Information Management	Manages user and device information.

Data Storage and Retrieval	Stores and retrieves calibration results, session records, and inference results.
----------------------------	---

5.2. Non-Functional Requirements

5.2.1 Reliability and Safety Requirements

- Since the system controls mobility devices based on user intentions, a high level of reliability and safety is required.
- If user intentions cannot be identified with sufficient confidence, the system shall maintain the REST state and refrain from generating movement commands.
- STOP commands shall be prioritized over all other movement commands to ensure user safety.
- In the event of sensor failures, communication disruptions, or abnormal signals, command generation shall be suspended and the system shall transition to a safe state.
- User-specific calibration data shall be utilized to minimize unintended operations caused by individual physiological differences.
- Device connectivity and command execution status shall be continuously monitored so that STOP or EMERGENCY_STOP commands can be applied immediately when abnormal conditions occur.

5.2.2 Data Format Requirements

- EMG data shall be represented in JSON format for communication between the ESP32 and the AI server.
- Data exchange between the AI server and the backend server shall also use JSON format.
- Inference results shall include fields such as Intent, Confidence, FatigueScore, and SignalQuality.
- The backend server shall maintain a structured data format for storing and managing inference results, control commands, and operation logs.

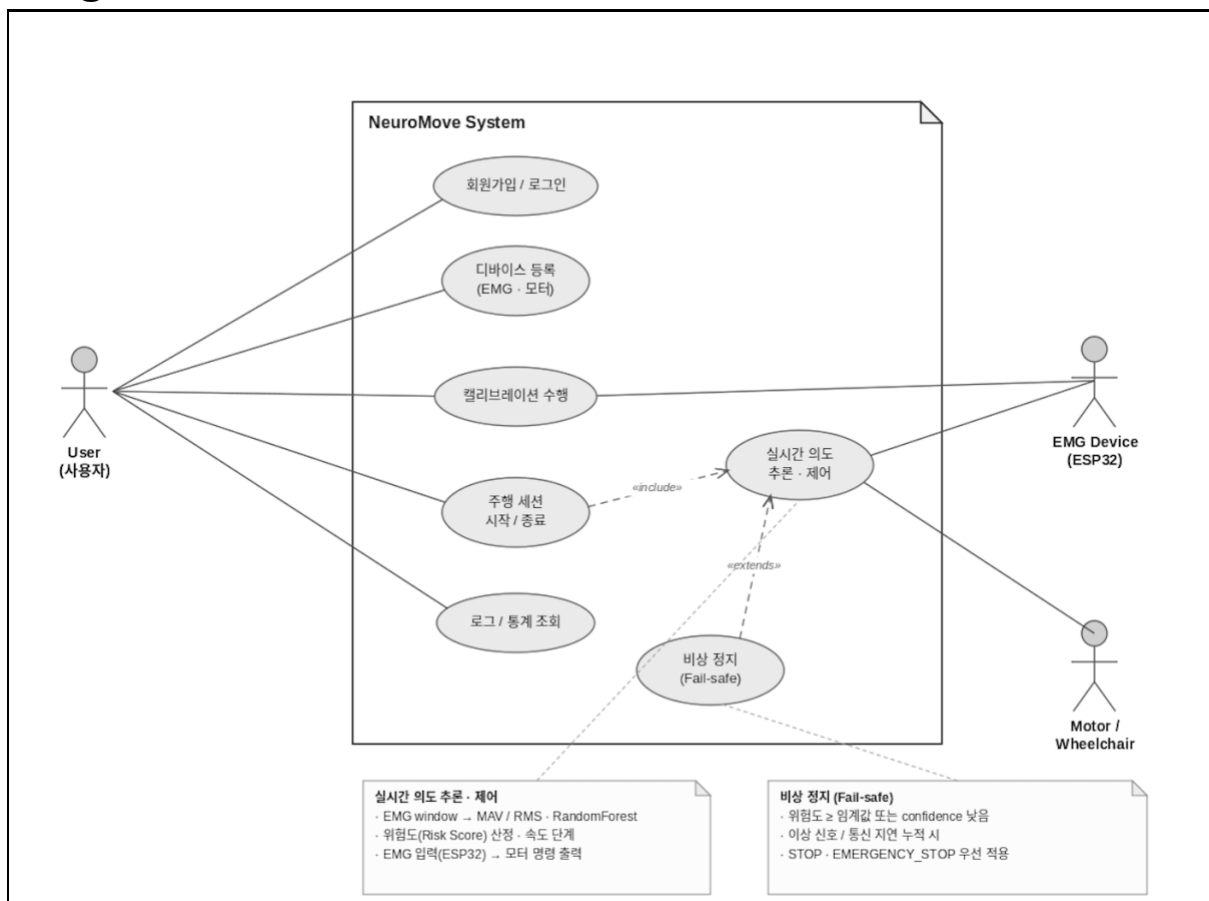
5.2.3 Input and Output Methods

- Input data are generated by converting analog EMG signals collected from MyoWare sensors into digital data through the ESP32.
- The ESP32 transmits data to the AI server in real time using WebSocket communication.
- The AI server preprocesses the received EMG signals, extracts relevant features, and classifies user intentions using the AI model.

- Classification results are transmitted to the backend server for storage and management, after which final control commands are generated.
- The backend system produces one of the four control outputs—LEFT, RIGHT, STOP, or REST—and is designed to support mobility assistance device control through integration with the ESP32.

6. System Design

6.1. Use Case Diagram



Primary Actors

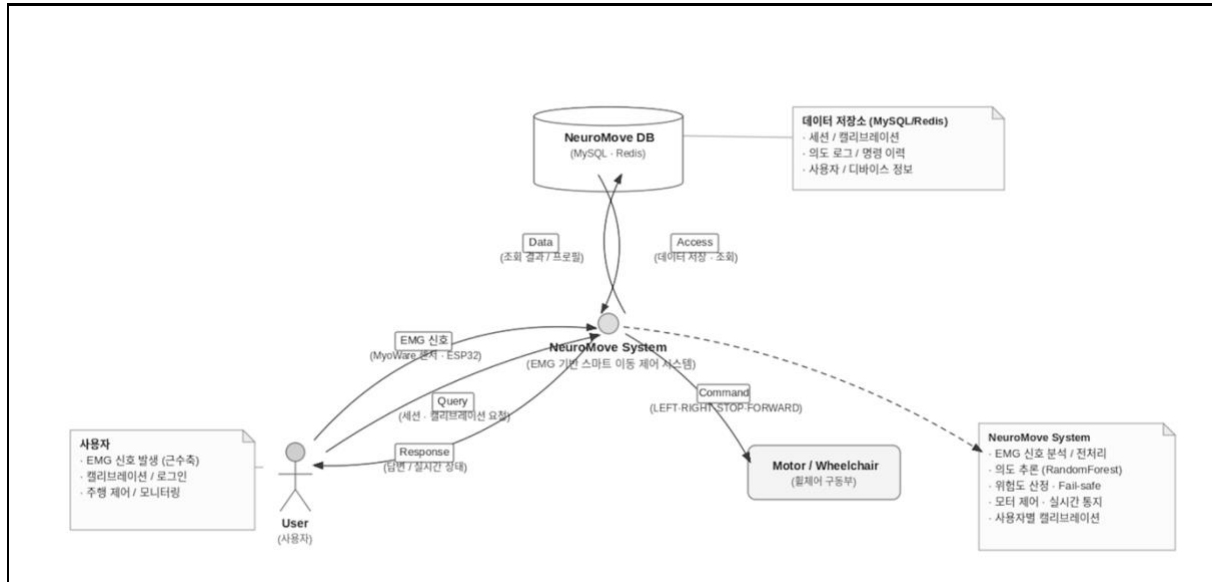
- **User:** The primary actor who performs account registration, device registration, calibration, driving session management, and log monitoring.
- **EMG Device (ESP32):** An external actor responsible for collecting EMG signals and transmitting them to the AI server in real time.

- **Motor Device (Mobility Assistance Device):** An external actor that receives control commands from the backend server and executes the corresponding physical movements.

Primary Use Cases

- **User Registration and Authentication:** Enables users to create accounts and manage access permissions through JWT-based authentication.
- **Device Registration and Onboarding:** Allows users to register and associate their EMG and motor devices with their accounts.
- **Calibration Process:** Collects EMG signals during the REST, LEFT, RIGHT, and STOP stages to generate user-specific baseline values and decision thresholds.
- **Driving Session Management:** Creates and terminates driving sessions, enabling real-time intention inference and control while generating session statistics upon completion.
- **Real-Time Intention-Based Control:** Generates mobility control commands based on AI inference results and transmits them to the motor device.
- **Log and Statistics Monitoring:** Provides access to session-specific intention logs, risk assessments, and command histories for analysis and monitoring purposes.

6.2. Data Flow Diagram



EMG data flows through the system as follows. The ESP32 packages three-channel EMG window data, including the sequence number, timestamp, sampling rate, window size, and channel values, into a JSON-formatted message and transmits it to the AI server through the WebSocket endpoint (/ai/stream/ws).

Upon receiving the data, the AI server identifies the current operating mode of the device (IDLE, CALIBRATION, or SESSION) based on the device ID and routes the incoming stream accordingly. During a driving session, recent EMG windows are accumulated, and

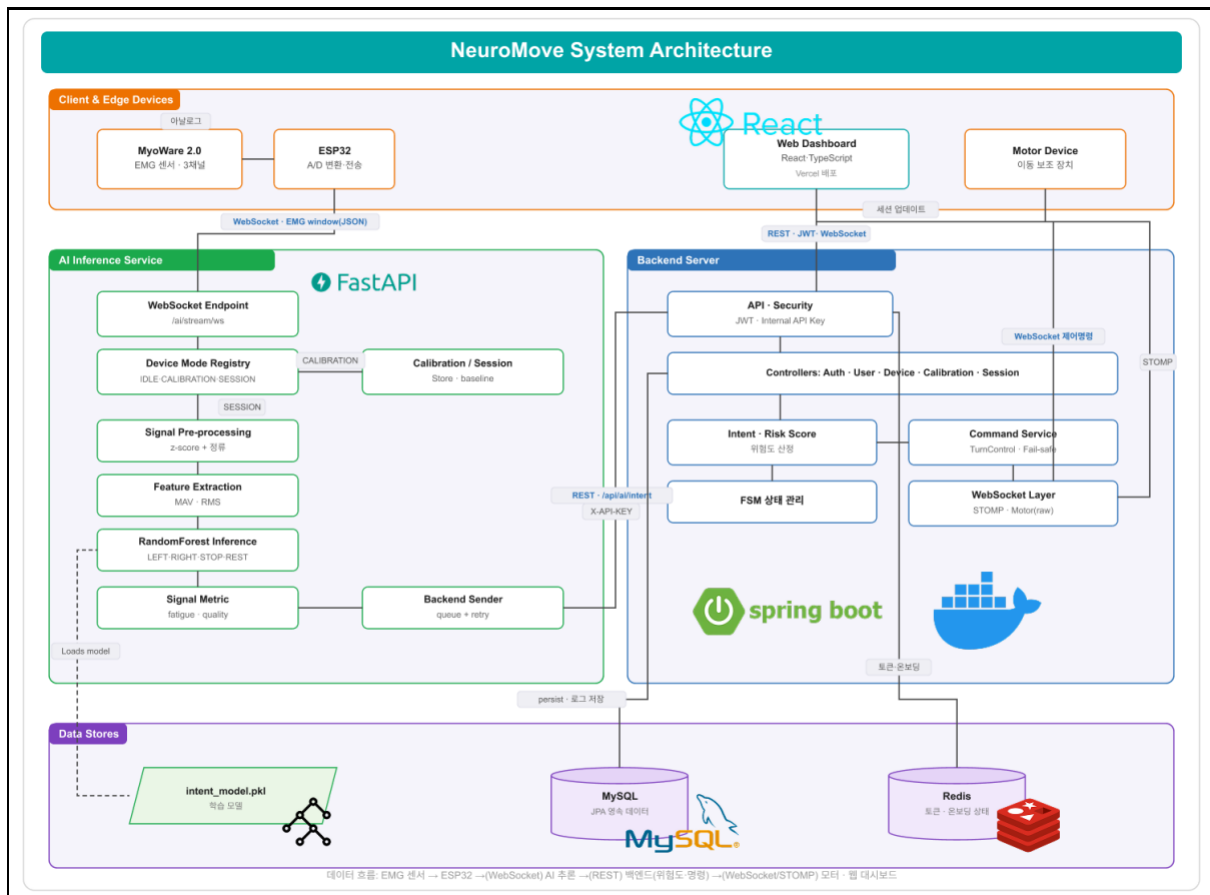
signal processing is performed after a predefined number of windows (five) have been collected. The processing pipeline includes signal preprocessing, such as DC offset removal or z-score normalization based on calibration baselines, followed by rectification. Subsequently, feature extraction is performed by calculating channel-wise Mean Absolute Value (MAV) and Root Mean Square (RMS) features. These features are then used as inputs to a Random Forest classifier for intention inference. At the same time, additional metrics including fatigue score and signal quality are computed.

The inference result, consisting of the predicted intent, confidence score, fatigue score, and signal quality, is transmitted to the backend server through a REST API endpoint (`POST /api/ai/intent`). To prevent blocking the real-time streaming pipeline, this transmission is handled asynchronously using a background queue and retry mechanism.

The backend server calculates a Risk Score based on the received intent, confidence score, fatigue score, and signal quality. It further validates the timestamp, confidence level, and signal quality before applying predefined fail-safe policies. Based on these evaluations, the server determines the appropriate Finite State Machine (FSM) state and speed level, and generates a final control command such as REST, LEFT, RIGHT, STOP, EMERGENCY_STOP, or BLOCKED.

Except for the BLOCKED state, generated control commands are transmitted to the motor device through WebSocket communication, resulting in actual motor operation. Simultaneously, AI inference results, risk scores, FSM states, and motor status information are delivered to the frontend dashboard via a STOMP-based WebSocket connection, allowing users to monitor system status in real time through a visual interface.

6.4. Architecture Diagram



NeuroMove follows a multi-tier architecture consisting of an Embedded Layer, AI Server Layer, Backend Layer, and Frontend Layer.

- **Embedded Layer:** Consists of MyoWare 2.0 EMG sensors and an ESP32 microcontroller. It converts analog signals into digital data and streams them via WebSocket communication.
- **AI Server Layer:** Built using Python, FastAPI, and Uvicorn, this layer is responsible for signal processing and intention inference. It utilizes a scikit-learn Random Forest model (`intent_model.pkl`) and processes signals using NumPy and Pandas. The Random Forest algorithm was adopted as the AI model for the real-time mobility control system because it provides stable classification performance even with relatively small EMG datasets and offers fast inference speed.
- **Backend Layer:** Implemented with Java 17 and Spring Boot 3.2.5, this layer is responsible for authentication, device, session, and calibration management, risk assessment, command generation, motor control, and real-time notifications. MySQL is used for data storage, while Redis is used for transient states such as onboarding information and tokens.
- **Frontend Layer:** A web dashboard built with React, TypeScript, Vite, and shadcn/ui. Real-time status updates are handled through STOMP-based WebSocket communication, while non-real-time requests such as user authentication, session management, and log retrieval are processed through REST APIs secured with JWT authentication. The frontend is deployed on Vercel.

The backend server runs in an AWS EC2 environment using Docker.

Internal communication between the AI server and the backend server is authenticated using an X-API-KEY (internal API key), while communication between users and the backend server is protected using JWT tokens.

7. Implementation

7.1. Design Methods

This project utilizes the following development environment for real-time biosignal processing and movement intention analysis.

- **Hardware:** MyoWare 2.0 EMG Sensor, ESP32
- **Embedded:** Arduino IDE, C++
- **AI Server:** Python, FastAPI, NumPy, Pandas, Scikit-learn
- **Backend:** Java, Spring Boot
- **Database:** MySQL
- **Communication:** WebSocket, REST API
- **Version Control:** Git, GitHub
- **Development Tools:** Visual Studio Code, IntelliJ IDEA
- **Deployment:** Docker

A Random Forest-based machine learning model is used for AI model training, with MAV (Mean Absolute Value) and RMS (Root Mean Square) employed as feature values.

7.2. Code explanation

The system consists of three repositories: the AI server, backend server, and frontend. The core modules and implementations of each repository are described below.

7.2.1 AI Server (NeuroMove_AI)

The AI server is implemented using Python and FastAPI and is responsible for EMG signal processing and user intention inference. It is organized into routers, services, schemas, storage, models, and training modules.

- **main.py:** Serves as the FastAPI application entry point and registers the session, calibration, and stream routers. Through the lifespan mechanism, a background sweeper is executed to periodically release locks for devices whose EMG streams have been disconnected for more than 30 seconds, thereby preventing lock leakage.

- **routers/stream_router.py:** Provides a single WebSocket channel (`/ai/stream/ws`) connected to the ESP32. The ESP32 transmits only EMG windows regardless of calibration or session mode, while the server determines the device mode based on the deviceId and processes the data accordingly.
- **storage/device_mode_registry.py:** Manages the current mode (IDLE, CALIBRATION, SESSION) and active ID for each device and serves as the basis for routing incoming data to the appropriate service.
- **services/signal_processing_service.py:** Removes DC offset from raw signals or performs channel-wise z-score normalization based on calibration baselines when available. Rectification (absolute value transformation) and optional normalization are then applied. The service supports three channels (ch0 and ch1 for directional control, and ch2 dedicated to STOP detection). After comparing multiple preprocessing methods, including DC offset removal, Min-Max normalization, and z-score normalization, the REST baseline-based z-score with rectification approach demonstrated the highest accuracy and was therefore adopted.
- **services/feature_service.py:** Computes MAV (Mean Absolute Value) and RMS (Root Mean Square) from the preprocessed channel signals and generates a six-dimensional feature vector in the form of `[ch0_mav, ch0_rms, ch1_mav, ch1_rms, ch2_mav, ch2_rms]`.
- **services/inference_service.py:** Classifies user intentions using the trained Random Forest model (`models/intent_model.pkl`). Before model inference, a rule-based safety mechanism is applied. If signal activation is below a predefined threshold, the result is classified as REST. If the dedicated STOP channel (ch2), attached to the user's jaw, exceeds its threshold, the system classifies the intention as STOP with higher priority than all other commands.
- **services/signal_metric_service.py:** Calculates signal quality (`signalQuality`) and muscle fatigue (`fatigueScore`) metrics, which are used by the backend server for risk assessment.
- **services/stream_service.py:** Accumulates session windows and performs inference every five windows. The results are placed into a background queue. A separate worker thread consumes the queue and transmits the results to the backend server, retrying up to three times to ensure that the streaming reception path remains unblocked.
- **training/train_intent_model.py:** Applies REST baseline-based z-score preprocessing, extracts MAV and RMS features, trains a `RandomForestClassifier(n_estimators=100, class_weight=balanced)`, and saves the trained model as `intent_model.pkl`.

For AI model training, several machine learning algorithms, including Decision Tree, Support Vector Machine (SVM), and Random Forest, were compared. Decision Trees offer a simple and interpretable structure; however, they are prone to overfitting and may exhibit unstable classification performance when applied to noisy EMG data with significant inter-user variability. SVM is a representative classification algorithm that provides strong performance even with relatively small datasets. However, it requires additional configurations for multi-class classification, is highly sensitive to hyperparameters such as C and Gamma, and involves relatively complex inference procedures that may increase computational overhead in real-time environments.

Deep learning models such as CNNs and LSTMs can learn complex EMG signal patterns effectively. However, achieving satisfactory performance generally requires large-scale training datasets and substantial computational resources. Since this project utilizes a limited amount of user data and requires real-time intention classification, deep learning models were considered inefficient in terms of both data availability and system resources.

In contrast, Random Forest mitigates overfitting by combining multiple Decision Trees through an ensemble approach and provides stable classification performance even in the presence of noise. Furthermore, it achieves high classification accuracy using only low-dimensional feature vectors such as MAV and RMS while maintaining fast training and inference speeds. In this project, MAV and RMS features extracted from three-channel EMG signals were used as inputs. Experimental results demonstrated more stable classification performance than both Decision Tree and SVM models. Considering the importance of real-time intention classification and a lightweight system architecture, RandomForestClassifier was ultimately selected.

7.2.2 Backend Server (NeuroMove_backend)

The backend server is implemented using Java 17 and Spring Boot 3.2.5. It follows a domain-driven structure consisting of the auth, device, calibration, session, intent, command, fsm, user, and websocket packages. The backend utilizes JPA (MySQL), Spring Security (JWT), Redis, WebSocket, and Swagger.

- **domain/auth:** Handles JWT-based authentication, user registration, login, token refresh, and onboarding. Temporary onboarding states are stored in Redis.
- **domain/intent:** Serves as the core domain that receives inference results from the AI server. The `IntentService` calculates the Risk Score using the formula:

$$0.4 \times \text{fatigue score} + 0.4 \times (1 - \text{signal quality}) + 0.2 \times \text{driving duration ratio}$$

It also determines the speed level (0–5) and generates the final control command.

- **Fail-safe Policy:** If commands become stale for more than three seconds and this condition accumulates three consecutive times, the system automatically issues a STOP command. If abnormal signal quality below 0.2 occurs three consecutive times, an EMERGENCY_STOP command is triggered. If confidence is below 0.5, the command is classified as BLOCKED. If the Risk Score exceeds 0.7, commands are blocked and the FSM state transitions to EMERGENCY_STOP, ensuring that safety remains the highest priority.
- **domain/command:** `TurnControlManager` is responsible for turning control. When LEFT or RIGHT is detected twice consecutively, the turn is confirmed and transmitted immediately. Approximately 500 ms later, a FORWARD command is automatically issued to prevent excessive turning. STOP and EMERGENCY_STOP commands are applied immediately.

- **domain/fsm:** Manages driving state transitions such as DRIVING, FATIGUE_COMPENSATING, and EMERGENCY_STOP, while recording state changes based on risk levels.
- **domain/calibration, domain/session, domain/device:** Manage user-specific calibration profiles, driving sessions, and EMG/motor device information. Integration with the AI server is handled through dedicated clients, namely [AiCalibrationClient](#) and [AiSessionClient](#).
- **domain/websocket:** Handles raw WebSocket communication with motor devices through [MotorWebSocketHandler](#) and STOMP-based real-time notifications to the frontend through [SessionWebSocketService](#). Internal calls from the AI server are authenticated using [InternalApiKeyFilter](#) (X-API-KEY).

Command	Generated By	Storage Location	Description
REST / LEFT / RIGHT / STOP	AI Inference Result	DB + Motor Transmission	Transmitted directly when classified as a normal command.
EMERGENCY_STOP	Backend	DB + Motor Transmission	Triggered when abnormal patterns are detected three consecutive times.
BLOCKED	Backend	Database Only	Motor transmission is blocked when confidence is low, commands are stale, or the risk score exceeds 0.7.
FORWARD	Backend	Motor Transmission Only	Automatically issued approximately 500 ms after a confirmed LEFT or RIGHT command to resume forward movement.
FINISH	Backend	Motor Transmission Only	Sent when a session ends to stop the motor device.

7.2.3 Frontend (NeuroMove_frontend)

The frontend is a Single Page Application (SPA) built with React, TypeScript, and Vite, providing a high-performance user interface and real-time data visualization. An intuitive user experience is implemented using shadcn/ui and Tailwind CSS, while continuous deployment (CD) is performed through Vercel.

- **Component-Based UI/UX (pages):** Consists of Login, Signup, SignupDevices, Main Dashboard, Logs, LogDetail, MyPage, and step-by-step calibration pages (Start, Step2–Step5, and Result).
- **Communication and State Management (lib/api.ts):** Applies state management based on TanStack Query and modularizes communication logic through a common API request function. JWT tokens are automatically attached to every request, and error handling for the backend's standardized response format (success/code/message/data) is centrally managed.
- **Real-Time Streaming Processing (lib/websocket.ts):** Implements a lightweight STOMP-over-WebSocket client without relying on external libraries. It maintains a persistent connection with the backend /ws endpoint and subscribes to /topic channels to receive session data such as intent, riskScore, command, and speedLevel in real time.
- **Data Visualization (components/Waveform.tsx):** Provides visualization of real-time signals and system status. Consistent layouts and smooth page transitions are achieved through AppSidebar and PageTransition components.

8. Testing

Test Case Name	SIGNAL_COLLECTION
Test Case ID	NEUROMOVE_TEST_1
Test Case Description	Verify that the EMG sensor can successfully collect the user's muscle signals and transmit them to the AI server.
Test Steps	1. Attach the MyoWare EMG sensor to the user's muscle. 2. Collect EMG signals through the ESP32. 3. Transmit the collected signals to the AI server using WebSocket communication. 4. Verify the number of channels, data length, and sequence numbers of the received data on the AI server. 5. Validate that the transmitted and received data are identical.

Expected Result	EMG signals are collected successfully. • Data are transmitted to the AI server without loss. • All channel data are stored correctly. • The system operates reliably in a real-time signal acquisition environment.
------------------------	---

Test Case Name	CALIBRATION
Test Case ID	NEUROMOVE_TEST_2
Test Case Description	Verify that user-specific calibration is performed correctly and that baseline values and thresholds are generated.
Test Steps	Start calibration and switch the device mode to CALIBRATION. 1. Collect EMG signals for the REST, LEFT, RIGHT, and STOP stages. 2. Calculate channel-wise means, standard deviations, and thresholds after each stage. 3. Verify that the calibration profile is saved when calibration is completed
Expected Result	• Signals are accumulated correctly for each stage. • Channel-specific baseline values (mean and standard deviation) and thresholds are generated. • The generated profile is applied during session inference.

Test Case Name	INTENT_INFERENCE
Test Case ID	NEUROMOVE_TEST_3
Test Case Description	Verify that the AI server accurately classifies user intentions from EMG feature vectors.

Test Steps	Start a driving session and switch the device mode to SESSION. 1. Input signals corresponding to LEFT, RIGHT, STOP, and REST actions. 2. Perform preprocessing, feature extraction (MAV and RMS), and Random Forest inference. 3. Verify that the predicted intent and confidence values match the expected results.
Expected Result	• Correct intentions are classified for each action. • STOP is prioritized when the dedicated STOP channel is activated. • Weak signals are classified as REST.

Test Case Name	FAIL_SAFE
Test Case ID	NEUROMOVE_TEST_4
Test Case Description	Verify that fail-safe policies prioritize safety commands under abnormal conditions.
Test Steps	Send inference results with confidence values below 0.5. 1. Send abnormal patterns with signal quality below 0.2 three consecutive times. 2. Generate stale command situations with timestamps delayed by more than three seconds three consecutive times. 3. Verify the final commands determined by the backend server for each case.
Expected Result	• Low-confidence results are processed as BLOCKED. • Three consecutive abnormal patterns trigger EMERGENCY_STOP. • Three consecutive stale commands trigger an automatic STOP.

Test Case Name	TURN_CONTROL
Test Case ID	NEUROMOVE_TEST_5

Test Case Description	Verify that the turning control logic correctly performs consecutive detection and automatic forward recovery.
Test Steps	Continuously send LEFT intentions. 1. Verify that a turning command is transmitted to the motor after two consecutive detections. 2. Verify that a FORWARD command is automatically transmitted approximately 500 ms later. 3. Verify that repeated execution of the same direction is suppressed when identical directions are continuously transmitted.
Expected Result	• A turn command is confirmed and transmitted once after two consecutive detections. • FORWARD is automatically transmitted after the turn. • Repeated execution of the same direction is blocked until FORWARD is issued.

Test Case Name	END_TO_END
Test Case ID	NEUROMOVE_TEST_6
Test Case Description	Verify that the entire pipeline, from EMG acquisition to motor control and dashboard display, operates correctly.
Test Steps	1. Transmit EMG signals from the ESP32 via WebSocket. 2. Verify that the AI server sends inference results to the backend server. 3. Verify that the backend server determines risk levels and commands and transmits them to the motor device and frontend. 4. Verify that intent, riskScore, and command values are updated in real time on the web dashboard.

Expected Result

- Commands are generated without data loss throughout the entire pipeline.
- Control commands are successfully delivered to the motor device.
- Real-time system status is displayed on the frontend dashboard.

9. Results

9.1. Results

Performance evaluation of the AI model and integrated system validation were conducted on the implemented system. The training dataset consisted of a total of 1,920 three-channel EMG windows, including approximately 480 samples each for LEFT, RIGHT, REST/NEUTRAL, and STOP/CLENCH. Seventy percent of the data was used for training, while the remaining thirty percent was used for testing.

The Random Forest classification model achieved a test accuracy of approximately **96.7%**. The class-wise Precision, Recall, and F1-score are presented below.

Class	Precision	Recall	F1-score	Support
LEFT	0.95	0.97	0.96	144
REST	0.95	0.94	0.95	144
RIGHT	0.97	0.98	0.98	145
STOP	0.99	0.97	0.98	143
Accuracy	-	-	0.97	576

In addition, integrated system validation confirmed the end-to-end operation of the entire pipeline. EMG streams transmitted from the ESP32 were processed by the AI server for intention inference, followed by risk assessment and command generation by the backend

server. The resulting commands were successfully delivered to both the motor device and the web dashboard in real time.

The AI server performed intention inference approximately every five windows (corresponding to the designed interval of approximately 0.5 seconds). Furthermore, the backend fail-safe mechanisms, including **BLOCKED**, **STOP**, and **EMERGENCY_STOP**, were verified to operate correctly in response to abnormal input conditions.

9.2. Result Analysis

Comparison of Preprocessing Methods and Rationale for Selection

To evaluate the impact of signal preprocessing on classification performance, a comparative experiment was conducted using the same training dataset (1,920 three-channel EMG windows), the same classifier (Random Forest, `n_estimators=100`), and the same training/testing split (70:30, `random_state=42`). The results are summarized below.

Preprocessing Methods	Accuracy	Macro F1-score
DC Offset Removal + Rectification	43.58%	0.435
Min-Max Normalization + Rectification	30.90%	0.310
REST Baseline z-score + Rectification (Selected)	96.70%	0.967

Simple DC offset removal or Min-Max normalization significantly reduced the absolute magnitude information contained in EMG signals when processing data on a window basis. In EMG-based intention classification, differences in activation levels across channels serve as important discriminative features. However, these preprocessing methods reduced the relative activation differences between channels, resulting in the loss of information required to distinguish movement directions. Consequently, classification accuracy dropped to approximately 30–44%, and misclassification between the LEFT and RIGHT classes occurred frequently.

In contrast, the z-score normalization with rectification approach, which uses the mean and standard deviation measured during the user's REST period as a baseline, preserved the activation differences between channels while incorporating user-specific signal characteristics. Because signals are interpreted relative to the user's baseline state, the

influence of individual differences such as sensor placement, skin condition, and muscle size can be reduced.

Experimental results showed that this method achieved the best performance, with an accuracy of 96.70% and a Macro F1-score of 0.967. Furthermore, the baseline information generated during calibration can be directly utilized during inference, making this approach practical as well as effective. Therefore, the z-score normalization with rectification method was selected as the final preprocessing technique for this project.

The experimental results demonstrated that the Random Forest model classified the four intentions—LEFT, RIGHT, STOP, and REST—with a high accuracy of approximately 96.7%. In particular, the STOP class achieved an F1-score of approximately 0.98 through the use of a dedicated STOP channel (ch2) combined with a rule-based priority processing mechanism. This indicates that the system can reliably recognize emergency stop intentions when users need to halt movement immediately. Since STOP commands are directly related to user safety in mobility assistance systems, the high STOP classification performance represents one of the most significant achievements of this project.

The LEFT and RIGHT classes also exhibited high precision and recall values. This performance is attributed to the design of independent muscle activation patterns for directional control and the effective extraction of channel-specific features. In particular, MAV and RMS features were found to be well suited for real-time systems, as they provide meaningful representations of muscle activation levels while requiring relatively low computational cost.

The REST class showed a comparatively lower recall value because minor muscle contractions and sensor noise may occur even when users intentionally remain inactive. Such signals can resemble weak activation patterns associated with movement intentions, resulting in occasional misclassification. To address this issue, an activation threshold was introduced so that signals below a predefined level are classified as REST. In addition, user-specific calibration data were utilized to minimize misclassification caused by individual variability.

Beyond intention classification, the proposed system employs a multi-layered safety architecture in which final control commands are determined based on a Risk Score. The Risk Score is calculated by considering multiple factors, including signal quality, muscle fatigue, and driving duration. Depending on the risk level, the system can adjust speed levels or trigger an EMERGENCY_STOP command. This design prevents a single inference error from directly resulting in unsafe movement commands.

Furthermore, multiple fail-safe policies were implemented. When confidence values are low, the system transitions to a BLOCKED state, while persistent degradation in signal quality automatically triggers a stop operation. These mechanisms proved effective in handling various real-world conditions, including sensor errors, communication delays, and accumulated user fatigue.

Overall, the experimental results demonstrate that the proposed EMG-based intention classification and risk-aware control framework can achieve a level of performance and reliability suitable for practical mobility assistance systems. Furthermore, the integration of

user-specific calibration and multiple safety layers highlights the potential of extending the system beyond simple intention classification toward a fully functional real-time mobility control platform.

Limitations and Future Work

- The current system focuses primarily on the implementation and validation of an EMG-based intention classification framework and a real-time control pipeline. Future work will involve collecting EMG data from a larger and more diverse group of users to further train the model and improve its performance.
- Since muscle characteristics and signal patterns vary among individuals, future improvements may include enhanced calibration procedures and adaptive control techniques that provide more personalized control.
- The current implementation has demonstrated the feasibility of real-time intention inference and control command generation. Future studies will integrate the system with actual mobility assistance devices to further evaluate response time and operational stability.
- In addition, usability studies involving users of different age groups and operating environments will be conducted to improve the convenience, reliability, and practical applicability of the system.

10. Team and Process

10.1. Design Schedule

The project was conducted during the first semester of 2026 (March–June). The major milestones and development schedule for each phase are summarized below.

Phase	Period	Major Activities	Deliverables
Requirements Analysis	March	Problem definition, literature review, requirements definition, technology stack selection	Project proposal, requirements specification

System Design	March–April	Architecture design, data format and API design, device configuration design	Design documents, API specifications
Hardware and Signal Processing	April	Integration of EMG sensors and ESP32, implementation of signal acquisition, preprocessing, and feature extraction	Signal processing module
AI Model Development	April–May	Data collection, feature design, Random Forest training and evaluation	Trained model, inference server
Backend and Frontend Development	April–May	Implementation of authentication, session, calibration, and command domains, development of web dashboard	Backend and frontend systems
Integration and Testing	May–June	End-to-end integration, fail-safe and turning control validation, bug fixing	Test results
Final Presentation and Documentation	June	Result analysis, report writing, presentation preparation	Final report, presentation materials

Phase	March				April				May				June		
	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15
Requirements Analysis															
System Design															
Hardware & Signal Processing															
AI Model Development															
Backend & Frontend Development															
Integration & Testing															
Final Presentation & Documentation															

10.2. Team Member Task Assignment

This project was carried out by a four-member team consisting of **Yeonwoo Lee, Sooyeon Park, Soobin Park, and Won Park**. Responsibilities were divided across the AI server, backend, frontend, and embedded hardware domains.

Team Member	Responsibility Area	Key Responsibilities
Yeonwoo Lee	AI Server / System Integration	Designed and implemented the AI server architecture, EMG signal processing pipeline, calibration workflow, and backend integration for real-time intent recognition.
Suyeon Park	AI Server / Hardware	Implemented EMG sensor data acquisition and real-time transmission; designed signal preprocessing and feature extraction (MAV, RMS); designed and trained the AI-based intention classification model.

Subeen Park	Backend / Hardware	Implemented Refresh Token authentication, onboarding integration, Redis-based device management, user status and operation log management; developed ESP32 firmware and motor control WebSocket integration; implemented timeout-based STOP and abnormal-pattern-triggered EMERGENCY_STOP handling.
Won Park	Backend / Frontend	Developed the web dashboard and calibration/session interfaces; implemented risk assessment and fail-safe logic; integrated frontend APIs and WebSocket (STOMP) communication; developed backend APIs for calibration, session, and AI inference domains; deployed the backend on AWS and the frontend on Vercel; conducted integrated end-to-end testing and quality assurance.

10.3. Budgets

The primary cost of this project is associated with the acquisition of EMG measurement hardware. During the development phase, local environments and free-tier services were utilized to minimize server operating costs. The figures below are estimates and may vary depending on suppliers and quantities purchased.

Item	Quantity	Estimated Cost (KRW)
MyoWare 2.0 EMG Sensor	3	Approximately 150,000 KRW
EEMG Electrode Pads (Consumables)	Multiple	Approximately 30,000 KRW per measurement session
ESP32 Development Board	1~2	Approximately 20,000 KRW
Motor / Drive Unit and Connection Components	1 Set	Approximately 50,000 KRW
Miscellaneous (Cables, Breadboards, etc.)	1 Set	Approximately 20,000 KRW

Server Operation (Local Environment / Free Tier during Development)	2	Approximately 40,000 KRW
Total (Estimated)	-	Approximately 310,000 KRW

In the future, if commercialization is pursued through integration with actual smart wheelchairs or mobility assistance devices, additional costs may be incurred for motor control units, battery systems, supplementary sensors, and cloud server infrastructure.

11. Conclusion

This project designed and implemented **NeuroMove**, a smart mobility control system that infers user movement intentions from electromyography (EMG) signals in real time and translates them into mobility assistance device control commands using low-cost EMG sensors and an ESP32 microcontroller. The system consists of three layers—an AI server (FastAPI), a backend server (Spring Boot), and a frontend application (React)—and implements an end-to-end pipeline covering signal acquisition, intention inference, risk-based command generation, motor control, and real-time visualization.

The AI model utilized MAV and RMS features together with a Random Forest classifier to classify four movement intentions—LEFT, RIGHT, STOP, and REST—with an accuracy of approximately 96.7%. Based on the inference results, the backend server performed risk assessment, FSM state management, and multiple fail-safe policies to improve the reliability and safety of the final control commands. In addition, real-time WebSocket communication between the ESP32 and the AI server enabled low-latency processing of EMG data, while a user-specific calibration mechanism was implemented to accommodate differences in individual muscle characteristics.

Particular emphasis was placed on safety through the integration of multiple protection mechanisms, including rule-based priority processing for the dedicated STOP channel, activation-threshold-based REST classification, risk assessment, and multi-layered fail-safe policies. Furthermore, turning control logic, device connection management, and session management functions were implemented to establish a system architecture capable of operating reliably in practical environments. Through this work, the feasibility of developing not only an intention classification model but also a real-time mobility control system was successfully demonstrated.

During the project, several limitations were identified, including variability in EMG signals across users, signal noise, and challenges in achieving robust generalization with limited training data. In particular, it was observed that EMG signal characteristics may vary depending on sensor placement and user condition, highlighting the need for additional data collection and model validation involving a wider range of users. Moreover, since the current validation was conducted in a simulation and prototype environment, further long-term

experiments integrated with actual mobility assistance devices are required. In addition, although EMG signals and control commands were successfully transmitted and processed, the low-cost motor kit used in the experiments exhibited limited assembly precision. Mechanical play in one of the wheels reduced the accuracy of straight-line movement and directional control during actual operation.

Future work will focus on retraining the model using real user EMG data, improving robustness, enhancing the calibration procedure, evaluating latency and safety in real motor-driven environments, and conducting usability studies involving diverse user groups. Furthermore, performance comparisons between the current Random Forest-based model and deep learning approaches such as CNNs and LSTMs will be investigated to develop more sophisticated intention recognition techniques.

The proposed system has the potential to improve mobility and autonomy for individuals with limited upper-limb functionality. It also serves as a foundational technology that can be extended to various application domains, including smart wheelchairs, rehabilitation robots, prosthetic control systems, and smart healthcare devices. Ultimately, this project presents a practical example of biosignal-based human-machine interface technology and demonstrates its potential for the development of user-centered intelligent mobility assistance systems.

Reference

- MyoWare 2.0 Muscle Sensor, SparkFun Electronics. <https://www.sparkfun.com/>
- Espressif Systems, ESP32 Technical Reference. <https://www.espressif.com/>
- FastAPI Documentation. <https://fastapi.tiangolo.com/>
- Spring Boot Reference Documentation. <https://spring.io/projects/spring-boot>
- scikit-learn: RandomForestClassifier. <https://scikit-learn.org/>
- React Documentation. <https://react.dev/>
- M. B. I. Reaz, M. S. Hussain, F. Mohd-Yasin, Techniques of EMG signal analysis: detection, processing, classification and applications, Biological Procedures Online, 2006.